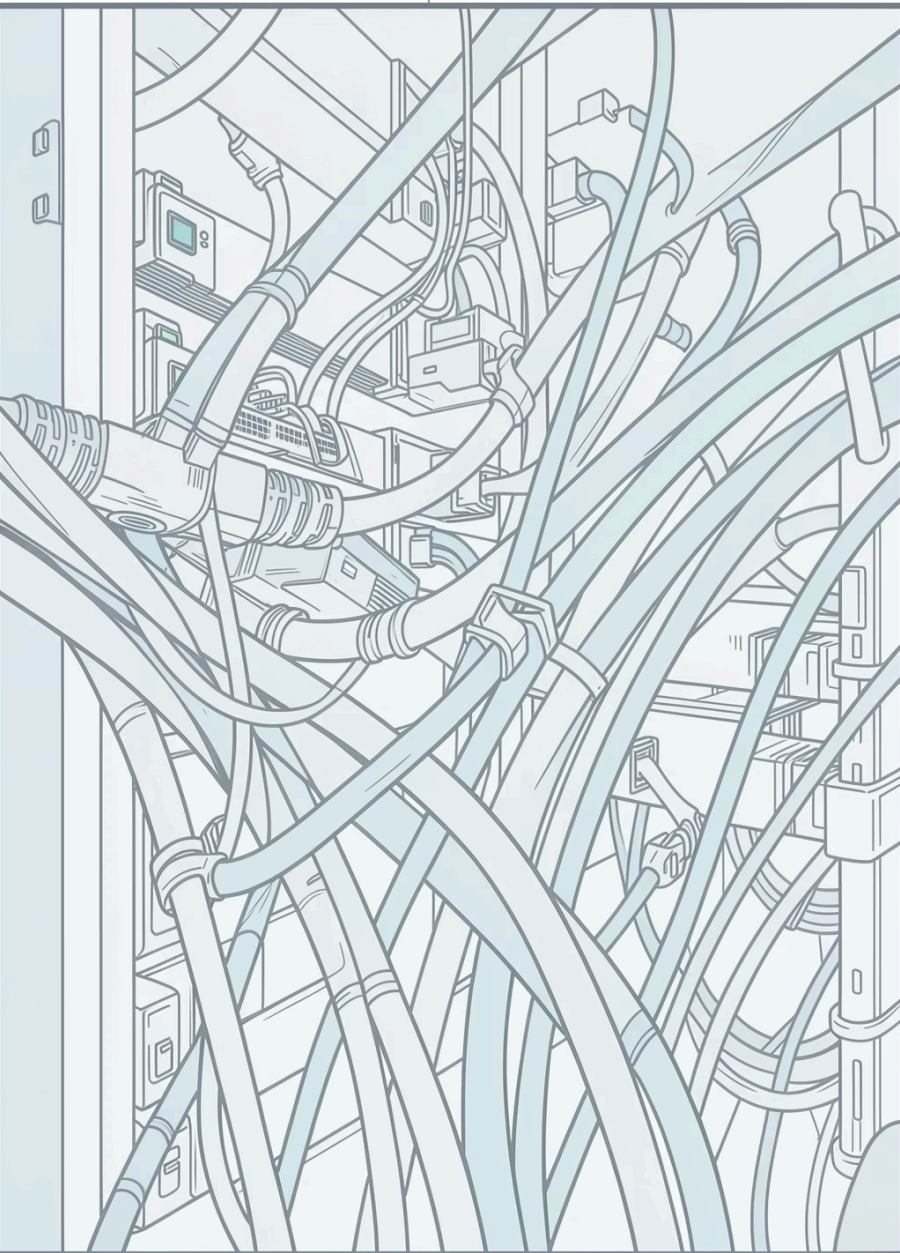




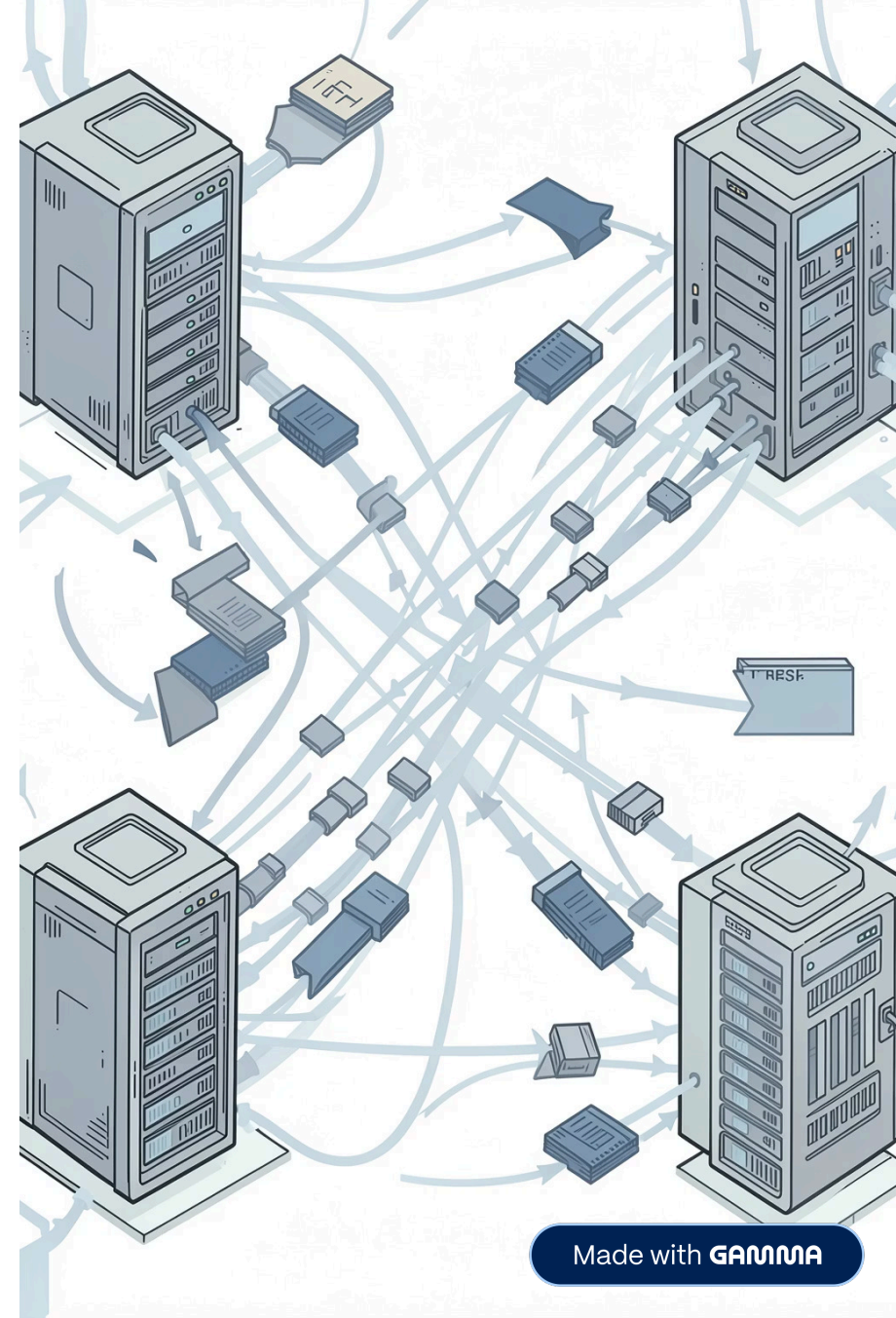
Zeek: IDS pasivo y simulación de ataques

Presentación académica — Arquitectura de sistemas y seguridad

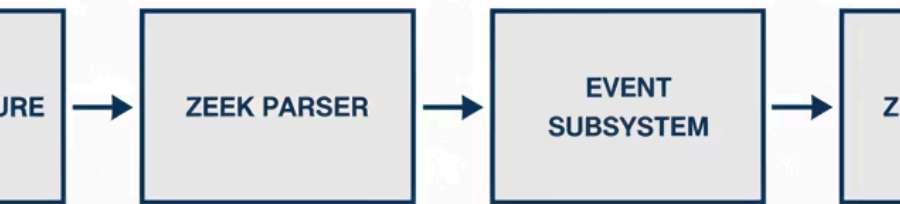
Nombres: Marco Pino, Matias Reyes, Joaquin Carranza, Juan Andino

Introducción a Zeek

- Zeek es un framework de análisis de tráfico de red orientado a eventos.
- Funciona como un IDS *pasivo*: monitoriza el tráfico, genera logs y alertas, no bloquea por defecto.
- Fuerte orientación a la visibilidad y análisis forense: construcción de registros ricos (conn.log, notice.log, weird.log).



DATA-DRIVEN ARCHITECTURE



¿Cómo funciona Zeek?

- Captura paquetes desde una interfaz (ej. loopback 'lo' / 127.0.0.1) usando libpcap.
- Desencadena una cadena de eventos (TCP connection, HTTP request, etc.) en tiempo real.
- Scripts escritos en el lenguaje de Zeek procesan esos eventos y generan logs/notices según la lógica definida.

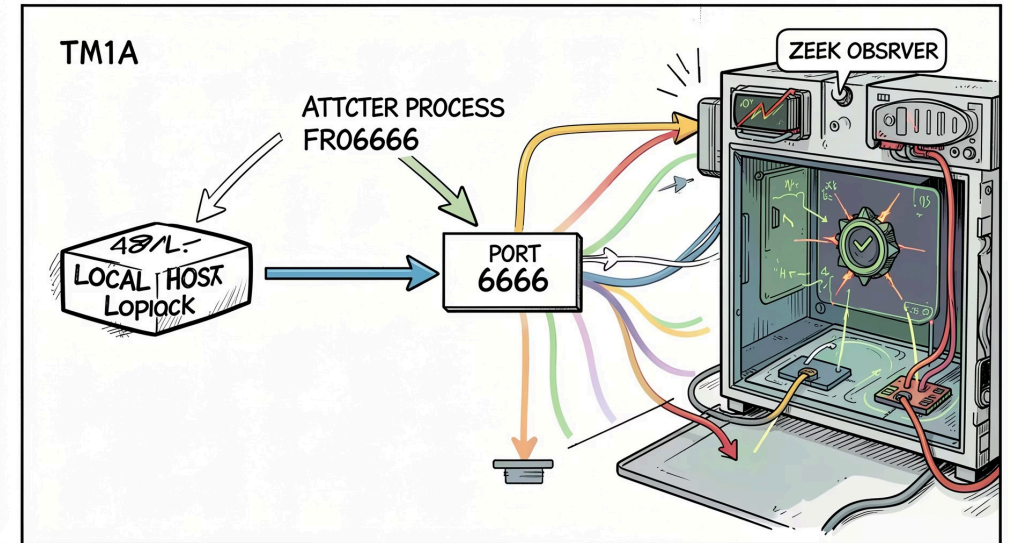


El diagrama resume el flujo: paquete → parsing → evento → script → log/alerta.

El laboratorio: Arquitectura del proyecto

Escenario:

- Red local simulada usando loopback 127.0.0.1.
- Atacante: máquina que ejecuta ataque.py (conexiones TCP al puerto 6666).
- Defensor: sensor Zeek en modo monitor que observa el tráfico en 'lo'.
- Objetivo: demostrar detección de intentos de conexión a puerto prohibido.



list.zeeek — la lista negra

Propósito: definir un conjunto (set) de puertos sospechosos y reutilizable por otros scripts.

```
# Fragmento ilustrativo (list.zeeek)
const suspicious_ports: set[port] = {
    6666/tcp,
    12345/tcp
};
```

Notas:

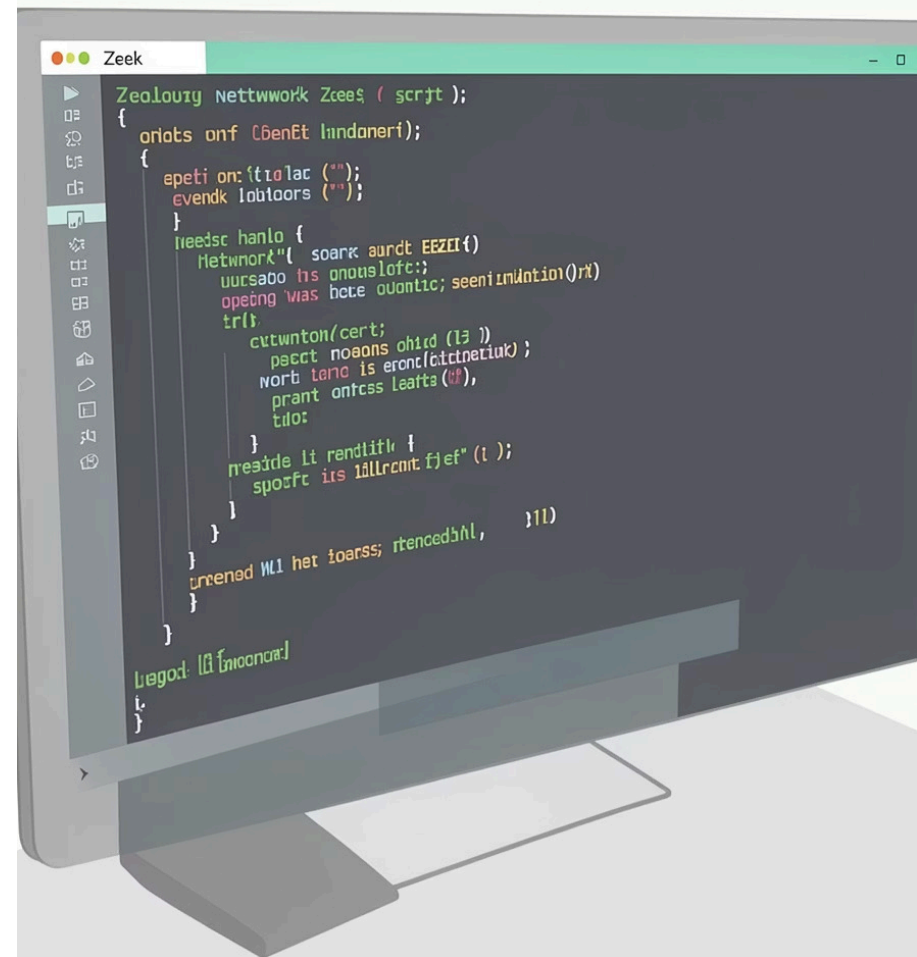
- Uso de `const` y `set[port]` facilita la modularidad y la actualización.
- Puede cargarse o redefinirse por configuración para distintos escenarios de laboratorio o producción.

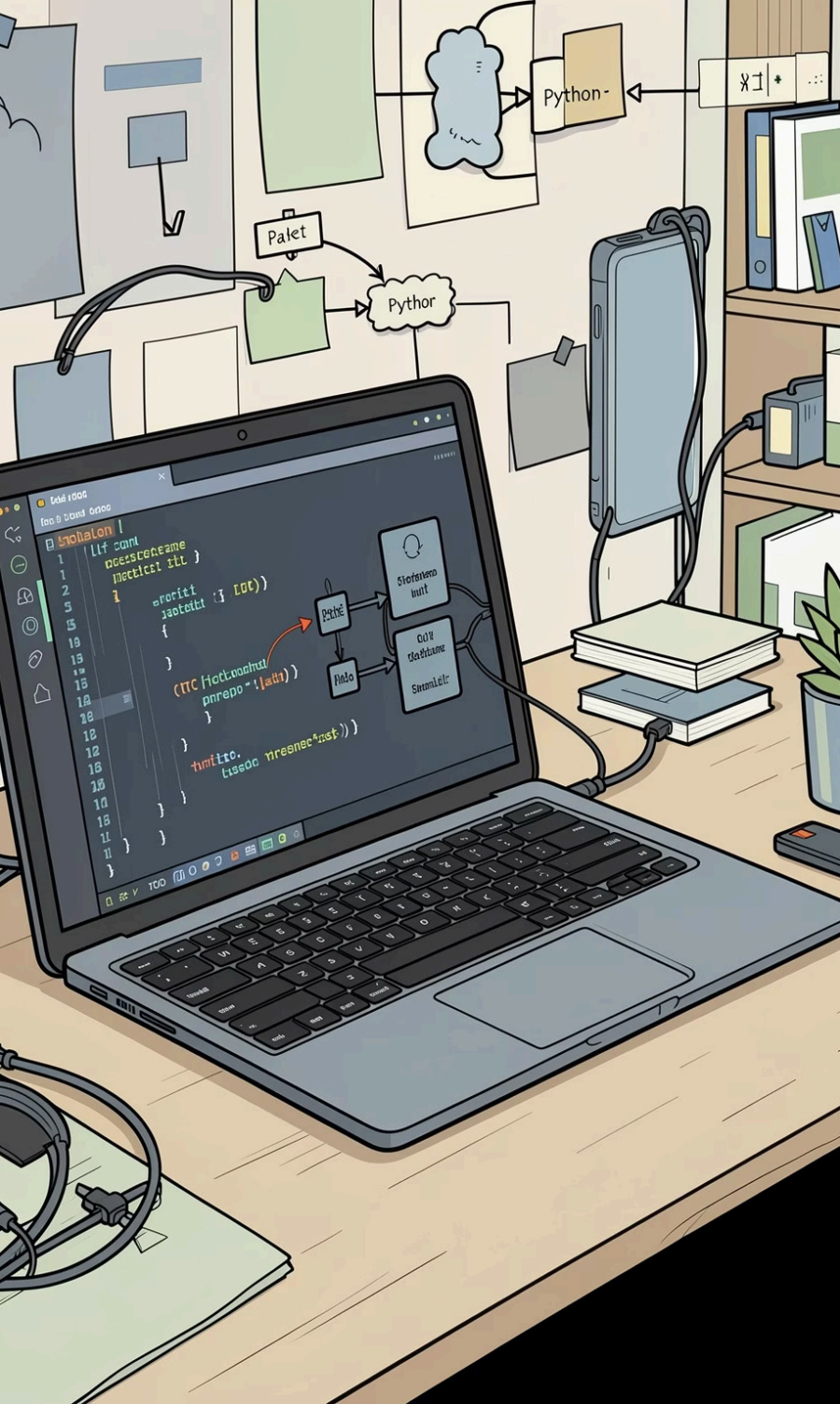
main.zeeK — motor de detección

Descripción y fragmentos clave del script principal que escucha nuevas conexiones y genera avisos.

```
# Fragmento ilustrativo (main.zeeK)
event new_connection(c: connection)
{
    local src = c$id$resp_h; # IP origen
    local dst_port = c$id$resp_p; # puerto destino
    if ( dst_port in suspicious_ports )
        NOTICE([$note="SuspiciousPort", $msg=fmt("Conexión a %s desde %s", dst_port, src)]);
}
```

- **Evento** `new_connection`: activado cuando Zeek detecta una nueva conexión TCP.
- Extracción de campos desde `connection`: IP origen (`resp_h`) y puerto destino (`resp_p`).
- Evaluación contra la lista negra y generación de `NOTICE` para alertas en pantalla y `notice.log`.





ataque.py — simulador de ataque

Objetivo: generar tráfico malicioso/controlado que provoque la regla de Zeek.

Fragmento ilustrativo (ataque.py)

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(1)
try:
    s.connect(("127.0.0.1", 6666))
except Exception:
    pass
s.close()
```

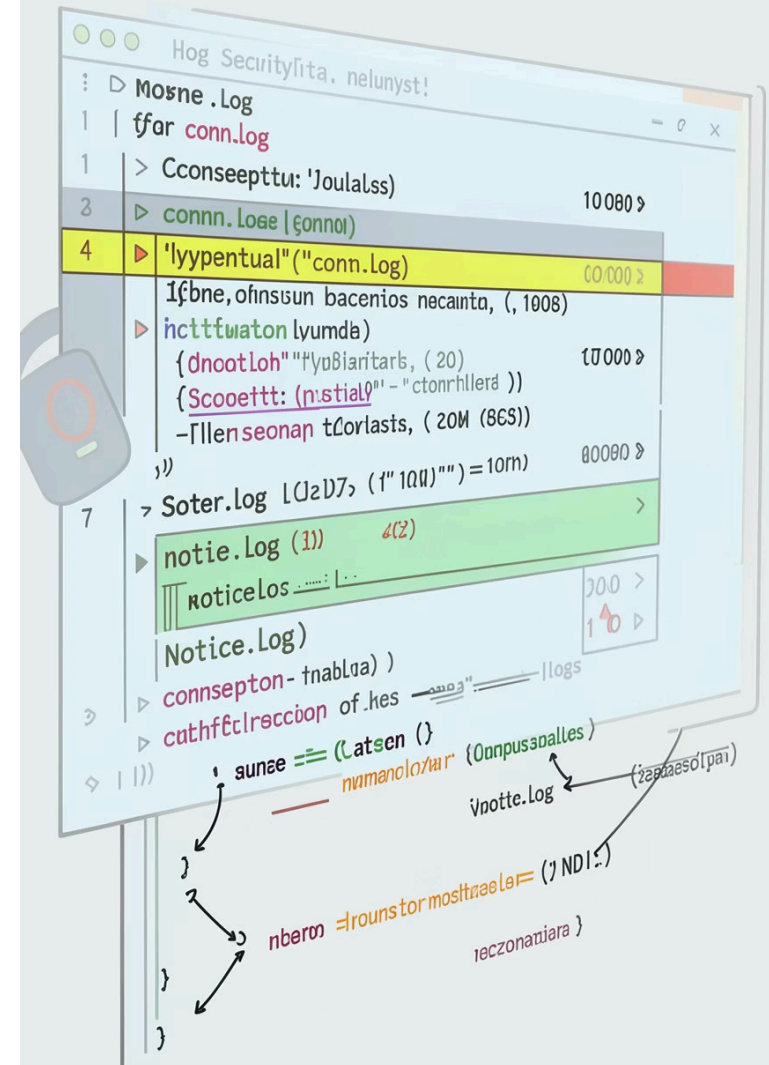
- Uso de la librería estándar `socket` para abrir intentos de conexión TCP (SYN).
- Permite repetir pruebas, variar puertos y generar tráfico para validación de detección.
- Se puede extender para enviar payloads o simulaciones más complejas.

Los logs de Zeek

- **conn.log**: registro estructurado de conexiones (5-tuple, duración, bytes transferidos).
- **notice.log**: avisos generados por NOTICE (alertas legibles por humanos y máquinas).
- **weird.log**: eventos anómalos o parsing inusual.
- En entornos profesionales, estos ficheros se exportan a un SIEM para correlación, alertas y respuesta.

Ejemplo breve (formato resumen):

```
# notice.log (ejemplo)
time uid note msg
... - SuspiciousPort Conexión a 6666/tcp desde 127.0.0.1
```



Buenas prácticas y consideraciones



Modularidad

Separar listas (list.zeeek) de la lógica (main.zeeek) facilita mantenimiento y pruebas.



Centralizar logs

Enviar conn.log y notice.log a un SIEM para correlación y alertas a gran escala.



Pruebas continuas

Automatizar ataques simulados (ataque.py) para validar reglas y evitar falsos negativos.



Entorno de monitorización

Monitorear interfaces relevantes, ajustar sensibilidad y documentar cambios en reglas.

D3EST PRACTIOYMERS



Conclusiones y próximos pasos

- Zeek ofrece una plataforma potente y extensible para visibilidad de red basada en eventos: ideal para análisis forense y detección.
 - Separar listas negras y motor de detección mejora la mantenibilidad y reutilización en distintos entornos.
 - Integración de Zeek con SIEM y pruebas automatizadas con scripts (ataque.py) son pasos imprescindibles para una defensa operativa.
 - Próximos pasos sugeridos: ampliar reglas (protocolos), enriquecer logs con metadata, y crear panel de alertas en SIEM.
-

Gracias — preguntas

 MARCO PINO

 MATIAS REYES

 JOAQUIN CARRANZA

 JUAN ANDINO